

Milestone 1 – Moving Grid

Student Information

Integrity Policy: All university integrity and class syllabus policies have been followed. I have neither given, nor received, nor have I tolerated others' use of unauthorized aid.

I understand and followed these policies: Yes No

Name:

Date:

Submission Details

Final **Changelist** number:

Verified build: Yes No

Required Configurations:

YouTubeLink:

Design Patterns completed and use in your code

(Demo in Video quickly each pattern – no show in video... no credit):

Singleton *(Show in SpriteBox manager)*

Factory *(Show how you are Creating Aliens)*

Object Pools *(Show in Image manage)*

Priority Queue *(Show TimerEventManager – how it was inserted priority order and early out)*

Proxy *(Show in SpriteProxy)*

Command *(Movement of Alien grid from a repeating timer commands)*

Iterator *(Drawing aliens in Sprite Batch Manager)*

Template *(In SpriteManager calling derived class from Base class)*

Composite *(Show the assembly of Aliens in a Grids and Columns)*

Null Object *(Show somewhere you using the concept of a null object)*

Adaptor *(Show in Texture class)*

Internal code

Labeling all news *(LTN – owner or STN – why)*

Object Pooling *(Prove its working)*

Discussion (What did you learn):

Verify Builds

- Follow the Piazza procedure on submission
 - Verify your submission compiles and works at the changelist number.
- Verify that only MINIMUM files are submitted
 - No – Generated files
 - *.pdb, *.suo, *.sdf, *.user, *.obj, *.exe, *.log, *.pdb, *.db, *.user
 - Anything that is generated by the compiler should not be included
 - No – Generated directories
 - /Debug, /Release, /Log, /ipch, /.vs
- Typical files project files that are required
 - *.sln, *.csproj, *.cs,
 - App.config, AssemblyInfo.cs, CleanMe.bat
 - Resources Directory:
 - *.tga, *.dll, *.wav, *.glsl, *.azul

Standard Rules

Submit multiple times to Perforce

- Submit your work as you go to perform several times (at least 5)
 - As soon as you get something working, submit to perforce
 - Have reasonable check-in comments
 - Points will be deducted if minimum is not reached

Submission Report

- Fill out the submission Report
 - No report, no grade

Code and project needs to compile and run

- Make sure that your program compiles and runs
 - Warning level 4
 - NO Warnings or ERRORS
 - Your code should be squeaky clean.
 - Code needs to work “as-is”.
 - No modifications to files or deleting files necessary to compile or run.
 - All your code must compile from perforce with no modifications.
 - Otherwise it's a 0, no exceptions

Project needs to run to completion

- If it crashes for any reason...
 - It will not be graded and you get a 0

No Containers

- Containers (No automatic containers or arrays)
- Template or generic parameters
- No arrays
 - You need to do this the old fashion way - **YOU EARNED IT**

Leave Project Settings

- Do NOT change the project or warning level
 - Any changing of level or suppression of warnings is an integrity issue

Simple C#

- No .Net
- We are using the basics
 - Types:
 - Class, Structs, intrinsic types (int, float, bool, etc...)
 - Basics language features
 - Inheritance, methods, abstract, virtual, etc...
 - No properties – set/get generated accessors

No Debug code or files disabled

- Make sure the program has only active code
 - If you added debug code or commented out code,
 - please return to code to active state or remove it

Adding files to this project

- Make sure you add the files in the appropriate sub-directories
- Make sure any new files are successfully integrated into the project
- Make sure your new files are submitted to Perforce

Due Dates

- See Piazza for due date and time
- Submit program perforce in your student directory assignment supplied.
- Fill out your this **Submission Report** and commit to perforce
 - **ONLY** use Adobe Reader to fill out form, all others will be rejected.
 - Fill out the form and discussion for full credit.

Goals

- Learn
 - Alien Grid moving
 - Moving - step by step
 - Aliens moving with lock step movement
 - Correct screen size and aliens proportion to final size
 - Look like the game
 - Black screen
 - Grid,Column,Aliens boxes displayed

Assignments

Grading:

1. Game Demo

- Final Screen Size, shape and proportionality
 - Black Screen, Green or White Aliens
- 55 Aliens (5 rows, 11 columns)
 - Squid 1 – rows
 - Crab 2 – rows
 - Octopus 2 – rows
 - **NEW sizes:**
 - Squid(x,y,w,h): (0, 0, 24, 25)
 - CrabA(x,y,w,h): (0, 0, 28, 25) ← changed from Sprint 4
 - Octopus(x,y,w,h): (0, 0, 36, 25)

2. Rendering

- Screen Size and Color
 - (width x height): 672 x 768 ← screen must be at this resolution
 - Clear Color : Black (RGB) (0,0,0) Alpha(1)
- Create a 11x5 Grid of aliens
 - Use the new *SpaceInvaders_ROM.tga*
 - *SpaceInvaders_ROM.txt* – Image dimensions are in this text file
 - Remember (in-case you forget)
 - Top row – squid
 - Middle 2 rows – crab
 - Bottom 2 rows – octopus
- Layout of Game Objects
 - GameObject (width x height)
 - Squid size: 24x 25
 - Crab size: 28 x 25 ← changed from Sprint 4
 - Octopus size: 36 x 25

- Game object are arranged
 - 50 pixels apart horizontally
 - 50 pixels apart vertically
 - Grid of 5 rows x 11 columns
 - Bottom left Alien (Octopus)
 - start at (86, 400) ← changed from Sprint 4

3. Movement

- Aliens moving horizontally back and forth across the screen
 - Each movement is a discrete step
 - On each movement the animation changes
 - Moves step by step...
 - All aliens animate at 0.5 seconds intervals
 - All aliens move at 0.5 seconds intervals
 - Do not move down.. that's later for Final submission
- All Aliens move total of 100 pixels horizontally
 - From the center line of the screen
 - -50 pixel to +50 pixels
 - Total of 100 pixel movements
 - Each alien moves +/- 4 pixels per Update() function
- All aliens move in unison together
 - Mimic this video:
 - <https://youtu.be/aO1ibP4N3ck>
 - Example of movement with animation
 - Actual arcade video movement is a little staggered...
 - We are not doing that.
- Movement Timing and animation is controlled by a timer using commands
 - Aliens cycling between 2 different images per alien type
 - Movement is controlled by timers... moving the aliens together.

4. Composite pattern

- Aliens are added together in a Grid, Column pattern
 - 1 Grid, 11 Column, 55 aliens
- Each column has
 - Top row – squid
 - Middle 2 rows – crab
 - Bottom 2 rows – octopus
- Need to see boxes with colors
 - Grid – Green color(0,1,0)
 - Column – Red color(1,0,0)
 - Alien – Yellow color(1,1,0)
- Use Sprite batches
 - SpriteBox batch – hold sprite boxes
 - AlienBatch – holds game sprites
- Add an enable flag to control draw mode
 - Be able to turn on/off the boxes with a Key press
 - just add an enable flag to control draw mode
- Mimic the video



5. Quick Video demo of all the patterns

- Just talk about it in Demo
 - Show a quick UML and/or code to prove the pattern exists
- Prove that you can
 - You identify the pattern
 - You show the pattern in use by quickly highlighting critical code
 - Use UML when appropriate
- Example (Expectation)
 - **Singleton** (*Show in Sprite manager*)
 - Total time 20-30 seconds...
 - Show the quick UML of sprite manager
 - “Here’s the UML of the Sprite manager...”
 - (open the Sprite manager UML)
 - It has a base manager
 - Sprite manager is derived from it “
 - “ Here is the singleton mechanics
 - (While showing code...)
 - You can see the static instance is held here...
 - You can see it being called here..
 - in Game:: Load Content()
 - (Showing a line of code where you are calling the manager.)”
 - Expectation 20 – 30 seconds per pattern
 - Quick and to the point
 - Show a key area to prove its there...
 - Not deeper than that
 - Do NOT try to bluff... be honest where you are at.
 - It is a quick laundry list...
 - You been using these in sprints... just prove you have it and understand its purpose.

6. Labeling all news (LTN – owner or STN – why)

- **For every new used in the game add a comment field**
 - Mark the new as LTN (long term new) or STN (short term new)
 - State the owner of the allocation in the comment
 - If it’s a STN – say why it’s a short term

- **Long Term New (LTN)**

- *Mark all you news with their ownership...*
- **Example: Inside Image manager**
override protected NodeBase derivedCreateNode()

```
{  
    // LTN- ImageManager  
    NodeBase pNodeBase = new Image();  
    ...  
}
```

- **Short Term New (STN)**

- Describe why it's a short term... can't be I didn't implement that manager
// STN - Factory is only used once, to create objects then goes away
// No need for keeping this long term
BirdFactory BF = new BirdFactory(SpriteBatch.Name.AngryBirds);

General guidelines:

- LOOK at the Illegal list on piazza
 - Do not use any C# operations that on the illegal list
 - YES – no lambdas, generics, templates, arrays, properties... etc
 - If you don't know... ask on Piazza

Development

- Store project in student directory in perforce
- Do your work in the supplied Sprint5 project
 - Feel free to add new files and methods to the project

Submission

- Submit your MS1 directory into perforce:
 - /student/<yourname>/MS1/...
 - You need to submit a complete C# project
 - Solution, project and C# files (whatever it takes to build the project)
 - Do not submit anything that is auto generated
 - Run the supplied CleanMe.bat before submission
 - Should cleanup files
 - Fill out the Submission report and submit that pdf to your student directory
- YouTube recording
 - Record approximately 8-10 minutes, do not exceed 15 min (HARD LIMIT)
 - Add your voice commentary
 - Showing the required features
 - No show... no points
 - Demo
 - All features with commentary
- **CRITICALLY important**
 - Code needs to compile "as-is"
 - Please do your verify correctly
 - Not compiling 0- grade

Validation

Simple checklist to make sure that everything is submitted correctly

- Is the project compiling and running without any errors or warnings?
- Does the project runs **ALL** without crashing?
- Is the submission report filled in and submitted to performce?
- Follow the verification process for performce
 - Is all the code there and compiles “as-is”?
 - No extra files

Hints

Most assignments will have hints in a section like this.

- This milestone is a summary of the material to this point
 - Through composite patterns
 - Composite pattern is the new stuff
 - Follow class examples

Troubleshooting

- Print, print, print
 - Draw diagrams to help you understand
- Have fun... this shouldn't be stressful
 - Slow and steady discovery and development will get you there.